

RailFlow: A Hybrid Approach to Dynamic Railway Traffic Control Using Proximal Policy Optimization and Congestion-Aware Graph Algorithms

Sudhanshu Shukla

Department of Computer Science and Engineering

Aditya Patel

Department of Computer Science and Engineering

Abstract—Delay propagation on high-density mixed-traffic railway corridors is governed less by the magnitude of the initiating disturbance than by the latency of the dispatching response to it. Thirty days of running history for trains 12003 (Swarna Shatabdi) and 12229 (Lucknow Mail) on the New Delhi–Lucknow corridor show punctual origin departures, 2 to 5 min mean deviation, followed by 38 min of mid-corridor delay accumulation, peaking at 48 min at Etawah, before corrective action is applied. This paper presents RailFlow, a hybrid traffic-control architecture coupling a discrete tick-based movement simulator, a congestion-aware dynamic graph, deterministic Dijkstra re-planning, and a Proximal Policy Optimization (PPO) agent. Section weights are time-costs of the form base traversal time multiplied by a congestion factor defined piecewise over the occupancy-to-capacity ratio, so a geometrically short section becomes arbitrarily expensive under saturation and unbounded when closed. A* search is rejected under this metric: a geometric heuristic is either vacuously weak or, once calibrated to realised speed, inadmissible. Every proposal passes a structural-validity and strict-improvement gate before commitment. Across six scenarios, five runs each, 600 ticks per run, on a 36-station corridor model, the gated controller reduced cumulative network delay by 49.0 % against an uncontrolled baseline, held 100 % on-time performance under dual track closure where the baseline reached 0 %, and resolved 154 congestion events with 19 automated actions, an 87.7 % reduction in dispatcher interventions. Decision latency falls from a 19 min human-reaction proxy to a single one-minute tick.

Index Terms—Railway traffic control, reinforcement learning, proximal policy optimization, dynamic shortest path, Dijkstra's algorithm, congestion modelling, discrete-event simulation, safe control, Indian Railways.

I. INTRODUCTION

Railway capacity on a shared-corridor network is consumed not only by the trains that occupy it but by the decisions that are not made while it is occupied. On mixed-traffic corridors, where superfast, express, passenger and freight services contend for

the same block sections under a single published timetable, a local perturbation propagates along the corridor at a rate determined by the interval between the perturbation and the dispatcher's response. Because the published path of every train is computed offline against nominal, empty-network traversal times, and is never revised in flight, the schedule offers no mechanism for absorbing a disturbance other than the slack deliberately built into it. Once that slack is exhausted, delay is transferred to every service that subsequently requires the affected section.

This paper addresses the specific case of the New Delhi to Lucknow corridor of Indian Railways, hereafter NDLS–LKO, and presents RailFlow, a control architecture that closes the decision loop at the granularity of one simulated minute.

A. The NDLS–LKO Bottleneck

The corridor modelled in this work comprises 36 stations and 42 bidirectional track sections spanning 1757 route-kilometres, covering the two principal alignments between the Delhi and Lucknow terminals. The northern alignment runs NDLS–GZB–MB–BE–SPN–HRI–LKO by way of Moradabad and Bareilly; the southern alignment runs NDLS–ALJN–TDL–ETW–CNB–LKO by way of Aligarh, Tundla and Kanpur. The station set comprises two terminals, three major stations, 17 junctions and 14 minor halts. Section capacity, expressed as the number of trains that may occupy a section concurrently without degradation, ranges from one to four, and nominal section speeds range from 45 to 85 km/h.

The operational signature of this corridor is not uniform lateness but mid-corridor accumulation followed by partial recovery. Train 12003 departs Lucknow Junction at a mean deviation of 5 min and reaches Etawah at a mean deviation of 48 min, with only 6.7 % of observed services on time at that point; it then recovers to 21 min at New Delhi. Train 12229 departs Lucknow at 2 min and reaches Rampur at 42 min, recovering to 20 min at New Delhi. Delay is therefore injected in the interior of the corridor, where section contention is highest, and is subsequently absorbed by terminal-approach slack. Punctuality measured only at the destination conceals the disturbance entirely.

The destination-delay distribution is additionally heavy-tailed. For train 12003 the mean destination delay over 30 observations is 21.0 min against a median of 4.0 min, with a standard deviation of 41.7 min and a maximum of 168 min. For train 12229 the mean is 19.4 min against a median of 1.0 min, with a standard deviation of 78.4 min driven by a single 438 min observation. Mean-based punctuality reporting is consequently a poor summary of this corridor: the typical service is close to schedule, and the aggregate statistic is dominated by a minority of severely disrupted days. It is precisely those days that a control system must address.

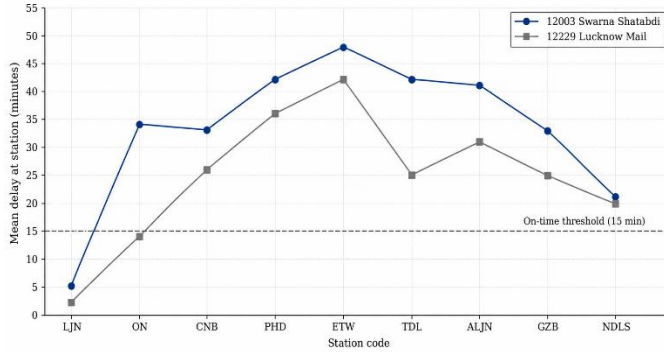


Fig. 1. Station-wise mean delay for trains 12003 and 12229 over a 30-day observation window. Both services depart their origin close to schedule and accumulate delay in the interior of the corridor before partial recovery at the destination approach.

B. Why Manual Dispatching Fails

The failure of manual dispatching on a corridor of this density is structural rather than a matter of operator competence. Four distinct mechanisms contribute.

1) *The underlying problem is a shortest-path computation on a time-varying graph.* The quantity a dispatcher must optimise is not distance but expected traversal time, and expected traversal time on a section is a function of how many trains currently occupy it. The weight of every edge in the network therefore changes on every tick of the clock. Resolving a corridor-wide shortest-path problem at that frequency, for every train in motion, is not a task a human operator can perform.

2) *Intervention is reactive rather than anticipatory.* A dispatcher acts on a delay that has already materialised and been reported. The measured corridor profile shows delay accumulating to a mean of 38 min before the response is applied. Every minute of that interval transfers additional delay to following services, so the cost of the latency compounds rather than remaining constant.

3) *The corridor-wide optimum is not decomposable into local decisions.* Granting precedence to a superfast service at one junction changes the occupancy, and hence the traversal cost, of sections many blocks away. A dispatcher working a single control area optimises locally, and the aggregate of locally optimal precedence decisions is not in general a globally optimal plan.

4) *The published path is never revised.* Timetable paths are computed against nominal traversal times on an empty network. When a section is closed by a signal failure or a derailment, the path is not recomputed; the train is held at the last feasible station until the section reopens or until a manual diversion is authorised. The closure scenario evaluated in Section IV isolates exactly this failure mode.

C. Contributions

This work makes the following contributions.

1) *A congestion-aware dynamic graph formulation* in which section weights are time-costs scaled by a piecewise congestion factor over the occupancy-to-capacity ratio, with an explicit discontinuity at saturation that renders an over-subscribed section strictly unattractive to the planner (Section III-A).

2) *A formal justification for rejecting A^* under this metric*, distinguishing the case in which a geometric heuristic remains admissible but vacuously uninformative from the case in which it is calibrated to realised speed and becomes inadmissible (Section III-B).

3) *A PPO formulation* of corridor dispatching with a 16-element bounded observation tensor and a four-element discrete action space covering the no-operation, reroute, hold and release decisions available to a dispatcher (Section III-C).

4) *A two-tier shielded controller* in which a learned or heuristic proposal is admitted only after passing a structural-validity and strict-improvement gate, bounding the worst-case behaviour of the learned policy by that of the deterministic tier (Section III-D).

5) *An empirical evaluation* against a real-world delay baseline derived from 60 daily observations of two named services, reporting delay reduction, intervention reduction and decision latency, together with an explicit account of the scenario in which the controller regresses (Section IV).

II. RELATED WORK

A. Operations Research Formulations

Real-time railway rescheduling has been studied extensively as a mathematical programming problem. Törnquist and Persson [1] formulate n-tracked traffic rescheduling during disturbances as a mixed integer linear program and evaluate several restrictions of the general model; Corman et al. [2] treat train rerouting during operations as a job-shop scheduling problem solved by tabu search over alternative graphs; Cacchiani et al. [3] survey the broader family of recovery models. These formulations are exact, admit provable optimality on the model as stated, and remain the reference point for offline timetable recovery.

Two properties make them unsuitable for the control loop targeted here. First, solver runtime is strongly sensitive to instance size and to the length of the rescheduling horizon; the literature reports solution times in the tens of seconds to minutes for realistic instances, which does not fit inside a control budget of one minute for a corridor of this extent while

leaving margin for actuation. Second, and more restrictively, an MILP requires the disturbance to be fully specified before the model is built. The formulation optimises against a declared scenario, so each change in network state, whether a new closure, a lifted speed restriction or a revised occupancy, requires the model to be rebuilt and re-solved. The control problem in this work is instead continuous: the state changes every tick and the controller must emit a decision on every tick.

B. Graph Search Baselines

Uninformed breadth-first search is not applicable because it minimises the number of sections traversed rather than the time to traverse them. On this corridor the two objectives are actively opposed: the southern alignment through Kanpur has a greater hop count than the northern alignment through Bareilly, yet it is the faster path whenever the northern alignment is congested or closed. A hop-count-optimal path is therefore not merely suboptimal but systematically wrong under the conditions this system is designed to handle.

Dijkstra's algorithm [4] is correct for the weight function defined in Section III-A, which is non-negative and finite on every traversable section. A^* [5] reduces the expanded set relative to Dijkstra when supplied with an admissible and informative heuristic. Section III-B shows that no geometric heuristic satisfies both conditions simultaneously on a graph whose weights are congestion-scaled, which is the basis on which A^* is rejected in this design.

C. Reinforcement Learning for Traffic Control

Reinforcement learning has been applied to railway rescheduling directly. Šemrov et al. [6] apply Q-learning to train rescheduling on a single-track railway and report improvement over the dispatching rules used as a baseline. In the policy-gradient family, Proximal Policy Optimization [7] has become a default choice for discrete-action control problems because its clipped surrogate objective provides the stability of trust-region methods at first-order cost. Standard treatments of the underlying formalism are given by Sutton and Barto [8], and the implementation used here is that of Stable-Baselines3 [9] operating against a Gymnasium environment [10].

The obstacle to deploying a learned policy in a safety-relevant setting is that nothing in the training objective prevents the policy from emitting an action that is structurally invalid or that degrades the objective it was trained on, particularly in states poorly covered by the training distribution. Alshiekh et al. [11] formalise the remedy as shielding: a correctness monitor is composed with the policy so that only actions satisfying a specification reach the environment. The gate described in Section III-D is a shield of this kind, specialised to the dispatching domain by requiring both structural validity and a strict improvement in projected delay.

D. Positioning of This Work

RailFlow occupies the space between the three lines above. It retains the graph-theoretic core of the operations research

formulations but replaces the batch solver with a per-tick Dijkstra re-plan that is cheap enough to execute inside the control budget. It adopts the learned-policy approach for the decision of when and on which train to act, rather than for the construction of the route itself, which remains deterministic. And it constrains the resulting composite with an explicit shield, so that the deterministic tier defines a floor on the behaviour of the system as a whole.

III. PROPOSED METHODOLOGY

RailFlow is organised as four cooperating components: a discrete tick-based movement engine that advances network state, a dynamic weighted graph that prices every section in units of time, a routing layer that re-plans against the current pricing, and a control layer that decides which train to act on and how. The control layer is itself two-tiered, comprising a proposal stage and a validation stage. Fig. 2 shows the resulting data flow.

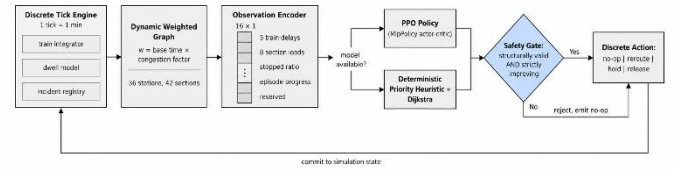


Fig. 2. RailFlow control architecture. The proposal stage is either the learned policy or the deterministic heuristic; both are routed through the same validation gate, so the gate defines the admissible action set regardless of which tier produced the proposal.

A. Discrete Tick Simulation and Graph Physics

1) *Temporal discretisation.* Network state advances in fixed ticks of one simulated minute. A tick is the atomic unit of both physics and control: every train position, every occupancy count and every control decision is resolved exactly once per tick. This choice fixes the control latency of the system at one minute by construction and makes delay, measured in minutes, directly commensurable with tick count.

2) *Train state and fractional movement.* A train is represented by its current node, its next node, the identifier of the section it occupies, a scalar progress variable on the interval $[0, 1]$ measuring the fraction of that section already traversed, a status drawn from {scheduled, moving, dwelling, stopped, arrived}, and an accumulated delay in minutes. Movement is fractional rather than node-to-node: within a tick the engine converts the train's effective speed into kilometres per minute, compares the distance available in the remaining tick budget against the distance to the next node, and either advances the progress variable or resolves an arrival and carries the unused remainder of the tick forward into the following section.

Carrying the remainder forward matters on a corridor that mixes long inter-junction runs with short halts: discarding it would introduce a systematic bias in which every station arrival silently consumes the balance of a tick, inflating journey times

in proportion to the number of intermediate stations rather than to distance. The carry loop is bounded at eight sections per tick, which prevents a degenerate graph from producing an unbounded iteration while remaining far above the maximum number of sections any train clears in one minute on this corridor.

3) *Occupancy and the congestion multiplier.* The base traversal time of section e , in minutes, follows from its length and its governing speed limit:

$$\tau(e) = \lceil d(e) / v_lim(e) \rceil \times 60 \quad (1)$$

where $d(e)$ is section length in kilometres and $v_lim(e)$ is the effective speed limit, taken as the section's nominal average speed unless a temporary restriction imposes a lower value, in which case the lower value governs. Let $n(e, t)$ denote the number of trains occupying section e at tick t and $C(e)$ its declared capacity. The load ratio is

$$\rho(e, t) = n(e, t) / \max(1, C(e)) \quad (2)$$

and the congestion multiplier applied to the base traversal time is defined piecewise over that ratio:

$$\mu(\rho) = \begin{cases} 1, & \rho \leq 0.7 \\ 1 + 1.4(\rho - 0.7), & 0.7 < \rho \leq 1.0 \\ 2\rho, & \rho > 1.0 \end{cases} \quad (3)$$

The three regimes encode three distinct operating conditions. Below 70 % occupancy the section is in free flow and imposes no penalty. Between 70 % and nominal capacity the penalty rises linearly, reaching a factor of 1.42 at exactly capacity. Above capacity the multiplier switches to twice the load ratio, which evaluates to 2.0 at the saturation point.

The discontinuity at $\rho = 1$ is deliberate and is the most consequential design decision in the weight function. The multiplier jumps from 1.42 to 2.00, a step of 41 %, at the instant a section becomes over-subscribed. A continuous formulation would allow the planner to treat marginal over-subscription as marginally more expensive, and the planner would then route trains onto saturated sections whenever the alternative was slightly longer. The step makes over-subscription a categorical rather than a gradual cost, so that any feasible alternative within a factor of two of the direct path is preferred to adding load to a section already at capacity. The resulting section weight is

$$w(e, t) = \begin{cases} \infty, & e \text{ blocked} \\ \tau(e) \cdot \mu(\rho(e, t)), & \text{otherwise} \end{cases} \quad (4)$$

A blocked section is assigned infinite rather than large finite weight. This is not a numerical convenience: it guarantees that the routing layer excludes closed sections from the reachable set entirely rather than selecting them when every alternative is sufficiently expensive, which is the behaviour required of a system that must never route a train onto a closed track.

4) *Effective speed.* Congestion is expressed twice in the model, once in the planner's cost function and once in the physics. A train T on section e moves at

$$v(T, e, t) = \min(V_T, v_lim(e)) \cdot k(\rho) \\ k(\rho) = 1 \text{ if } \rho < 0.7; \ 0.85 \text{ if } 0.7 \leq \rho \leq 1; \ 1/\rho \text{ if } \rho > 1 \quad (5)$$

where V_T is the class maximum speed of the train. The reciprocal form above capacity is the physically meaningful one: doubling the occupancy of a section halves the speed attainable on it, which is the behaviour of a block-signalled line in which following trains inherit the headway of the train ahead. The separation of the two expressions matters for evaluation: the planner is not given privileged access to the physics, and a route that the planner prices as cheap is not thereby made fast.

5) *Congestion delay charge.* Independently of the speed reduction, a train that begins a tick on an over-subscribed section accrues an explicit delay charge

$$\Delta_cong = \lceil \rho - 1 \rceil, \text{ charged once per tick when } \rho > 1 \quad (6)$$

with a floor of one minute. The charge is levied against the section the train occupies at the start of the tick and not once per section cleared within the tick, so that a train crossing several short sections in one minute is not penalised multiple times for a single minute of exposure.

6) *Dwell and class profiles.* Station dwell is the product of a station-specific base dwell and a class-specific multiplier, floored at one minute. Trains that do not halt at minor stations skip the dwell entirely at those stations. Table I gives the five class profiles. Priority is an integer on $[1, 5]$ and is the sole ordering key used by the deterministic controller when it selects which train to act on.

TABLE I
TRAIN CLASS PROFILES

Class	Priority	Max speed (km/h)	Halts at minor	Dwell factor
Superfast	5	120	No	0.8
Express	4	105	No	1.0
Passenger	3	80	Yes	1.3
Freight	2	65	No	1.8
Maintenance	1	40	Yes	1.5

Class profiles govern both the physics, through maximum speed and dwell, and the control policy, through priority. Priority 5 denotes the highest precedence.

7) *Incident taxonomy.* Disturbances are modelled in two families. Blocking incidents, comprising track closure, signal failure and maintenance block, set the section's blocked flag and therefore its weight to infinity. Speed incidents, comprising speed restriction and weather slowdown, impose a finite speed limit that supersedes the nominal section speed and so propagate into both (1) and (5). The distinction is material to the controller: a blocking incident removes a section from the reachable set and forces a topological re-plan, whereas a speed incident only re-prices it and may leave the original path optimal.

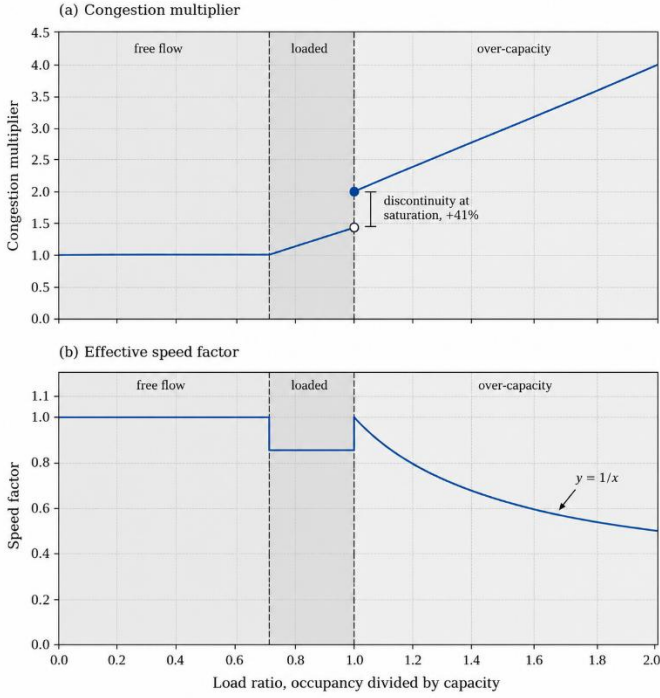


Fig. 3. The two channels through which occupancy enters the model. The planner sees the upper panel as a cost; the physics applies the lower panel as a speed reduction. The step at unit load ratio makes over-subscription a categorical cost rather than a marginal one.

B. Dynamic Dijkstra Routing versus A*

1) *The routing procedure.* Routing is a single-source shortest-path computation from the train's current or next node to its destination, evaluated against a snapshot of network occupancy taken once at the start of the tick. A binary heap is used as the priority queue. Sections whose weight evaluates to infinity are skipped during relaxation rather than being enqueued with a large finite key, so closed sections never enter the frontier. The cost of a complete path P is the sum of its section weights under the same occupancy snapshot:

$$W(P, t) = \sum w(e, t) \text{ for } e \in P \quad (7)$$

Taking a single occupancy reading per tick and evaluating every train's speed and every candidate route against that one reading is a correctness requirement rather than an optimisation. If occupancy were recomputed per train, the outcome of the tick would depend on the order in which trains were iterated, and the simulation would not be reproducible.

For this corridor, with 36 vertices and 42 edges, the binary-heap implementation runs in $O((V + E) \log V)$. The absolute cost of a single re-plan at this scale is negligible relative to the one-minute control budget, which is what permits the route to be recomputed from scratch on every decision rather than being incrementally repaired.

2) *Why A* is rejected.* A* [5] expands a strict subset of the nodes Dijkstra expands when its heuristic h is admissible, meaning $h(n) \leq h^*(n)$ for every node n , where h^* is the true remaining cost. The natural heuristic on a railway graph is geometric, derived from the great-circle or planar distance

between a node and the destination divided by some reference speed:

$$h_{\text{geo}}(n) = \lceil \text{dist}(n, \text{dest}) / v_{\text{ref}} \rceil \times 60 \quad (8)$$

The choice of v_{ref} forces a dilemma from which no setting escapes.

Case 1: v_{ref} set to the network maximum free-flow speed. The heuristic is then formally admissible, since congestion and speed restrictions only ever increase realised cost above the free-flow bound. But admissibility is purchased at the cost of informativeness. As load ratios rise, true remaining cost grows without bound through the $2p$ branch of (3) while h_{geo} remains fixed at its free-flow value, so the ratio h_{geo}/h^* tends to zero. The heuristic supplies progressively less discrimination exactly in the congested regime for which the system exists, and the expanded set converges on Dijkstra's. A* then pays the per-node cost of evaluating the heuristic and returns no pruning in exchange.

Case 2: v_{ref} calibrated to realised corridor speed. Raising the informativeness of the heuristic requires lowering v_{ref} towards the speed trains actually achieve. This makes h_{geo} larger, and the moment it exceeds the true cost of any free-flowing alternative the heuristic becomes inadmissible. A* then no longer guarantees an optimal path, and the specific failure it produces is the one this system must avoid: an inflated estimate on the free-flowing diversion causes the congested direct path to be expanded and returned in preference to it.

Two further properties of this graph independently undermine the geometric heuristic. First, under closure, geometric proximity is anti-correlated with reachability. When the Bareilly section is closed, the nodes geometrically nearest the destination along the northern alignment are precisely those that are unreachable, so the heuristic orders the frontier to expand into a dead end before considering the southern diversion that constitutes the answer. Second, the weights change on every tick, so any precomputed or landmark-based heuristic strong enough to be worth the trouble would require invalidation at the same frequency. The amortisation that justifies preprocessed heuristics on static road networks never occurs here.

The conclusion is that no geometric heuristic is simultaneously admissible and informative on a congestion-priced railway graph, and that at $V = 36$, $E = 42$ the asymptotic advantage A* would confer is in any case immaterial against a one-minute budget. Dijkstra's algorithm is therefore used unmodified, re-evaluated from scratch against live occupancy on every decision. Algorithm 1 states the procedure.

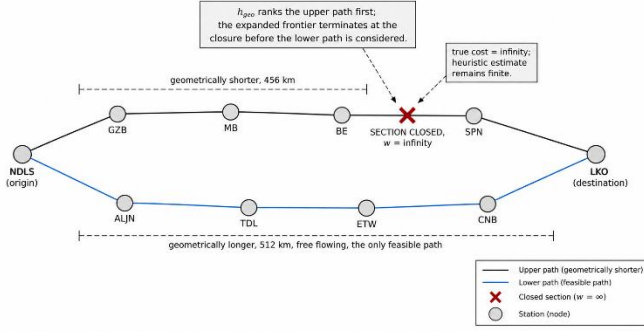


Fig. 4. Under closure the geometrically preferable alignment is the infeasible one. A distance-derived heuristic orders the search frontier towards the closed alignment, which is the ordering a dispatching system must not adopt.

Algorithm 1: Congestion-aware dynamic Dijkstra

Input : graph G , source s , destination z , occupancy map N
Output: least-time path $s \rightarrow z$, or NONE

```

1 for each  $v$  in  $V(G)$ :  $\text{dist}[v] \leftarrow \text{INF}$ ;  $\text{prev}[v] \leftarrow \text{NIL}$ 
2  $\text{dist}[s] \leftarrow 0$ ;  $Q \leftarrow \text{min-heap containing } (0, s)$ 
3 while  $Q$  not empty:
4    $(c, u) \leftarrow \text{extract-min}(Q)$ 
5   if  $c > \text{dist}[u]$ : continue // stale heap
6   if  $u = z$ : break // destination
7   for each  $(v, e)$  in  $\text{adjacency}[u]$ :
8      $\rho \leftarrow N[e] / \max(1, \text{capacity}(e))$ 
9      $w \leftarrow \text{INF}$  if  $\text{blocked}(e)$  else  $\tau(e) * \mu(\rho)$ 
10    if  $w = \text{INF}$ : continue // closed section:
never relax
11    if  $c + w < \text{dist}[v]$ :
12       $\text{dist}[v] \leftarrow c + w$ ;  $\text{prev}[v] \leftarrow u$ 
13      push( $Q, (\text{dist}[v], v)$ )
14 if  $\text{dist}[z] = \text{INF}$ : return NONE // z unreachable
this tick
15 return path reconstructed from prev

```

Line 10 is the guarantee that no returned path crosses a closed section, and line 14 is the signal on which the controller escalates: when the destination is unreachable under the current network state the correct control action is not a reroute but a hold, and the controller emits one.

C. PPO Reinforcement Learning Formulation

The routing layer answers where a train should go. It does not answer which train to act on, at what moment, or whether acting at all is preferable to waiting. That decision is formulated as a Markov decision process and approached with Proximal Policy Optimization [7].

1) *Observation space.* The observation is a one-dimensional tensor of 16 elements, each bounded on $[0, 1]$ and typed as 32-bit float. The encoding is as follows. Elements 1 to 5 carry the delays of the first five trains in the roster, normalised as $\min(1, \text{delay}/60)$, so that one hour of accumulated delay saturates the channel. Elements 6 to 13 carry the load ratios of the first eight sections, clipped as $\min(1, \rho)$. Element 14 is the fraction of trains currently in the stopped state, a scalar summary of how much of the network the controller has itself immobilised. Element 15 is normalised episode progress, tick divided by horizon, which supplies the policy with the temporal context required to distinguish an early-episode delay that can still be

recovered from a late-episode delay that cannot. Element 16 is reserved and zero-padded in the present version.

Bounding every channel on the unit interval removes the need for running normalisation statistics and makes the observation stationary across scenarios of differing scale. The cost is the truncation discussed in Section IV-G: the encoding observes a fixed-size prefix of the train and section collections rather than the whole network, which makes the observation dependent on collection ordering and does not generalise to rosters larger than five.

2) *Action space.* The action space is Discrete(4), chosen to mirror the four operations actually available to a dispatcher at a control desk:

a	Action	Semantics
0	No operation	Advance the tick without intervening
1	Reroute most delayed	Re-plan the highest-delay train by Dijkstra
2	Hold lowest priority	Stop the lowest-priority moving train
3	Release highest priority	Resume the highest-priority stopped train

Actions 1 through 3 are parameterised implicitly: the agent selects the kind of intervention and the environment resolves the target by an argmax over delay or priority. This keeps the action space constant as the roster size varies, which a per-train action space would not. It also confines the learned component to a genuinely strategic decision, since the route itself is always produced by the deterministic router and is never emitted by the policy.

3) *Reward.* The reward is the negation of a weighted sum of three penalties, evaluated on the post-transition state:

$$r_t = -[\sum_T \text{delay}_T + 10 \sum_e \max(0, \rho_e - 1) + 5 |S_t|] \quad (9)$$

where the first term is total accumulated delay over all trains, the second penalises over-capacity sections in proportion to the excess, and the third penalises the count of stopped trains $|S_t|$. The third term is what prevents the degenerate policy. Without it, stopping every train is attractive, because a stationary train on an empty network accrues no congestion penalty; the constant charge of 5 per stopped train per tick makes immobilisation a cost that must be repaid by a delay reduction elsewhere. The relative weights, 1 : 10 : 5, encode the operating preference that an over-subscribed section is worse than a minute of delay and that a deliberate hold is an intermediate cost.

4) *Episode structure and optimisation.* An episode is 240 ticks, four simulated hours, terminating on horizon expiry with no early termination condition. On reset the environment seeds the default mixed-peak scenario and releases the pause. PPO optimises the clipped surrogate objective

$$L(\theta) = E[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)] \quad (10)$$

with probability ratio $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ and $\hat{A}_t$ the advantage estimate. The clip prevents any single update from moving the policy far enough to invalidate

the trajectories it was estimated from, which is the property that makes PPO appropriate for an environment whose rollouts are expensive to collect. The implementation is the Stable-Baselines3 [9] MlpPolicy, a two-layer fully connected actor-critic, trained for 10 000 environment steps against the Gymnasium [10] interface. Table II summarises the configuration.

TABLE II
REINFORCEMENT LEARNING CONFIGURATION

Parameter	Value
Algorithm	PPO, clipped surrogate objective
Implementation	Stable-Baselines3 MlpPolicy
Observation space	Box(0.0, 1.0, shape=(16,), float32)
Action space	Discrete(4)
Episode horizon	240 ticks (4 simulated hours)
Training budget	10 000 environment steps
Tick resolution	1 tick = 1 simulated minute
Reward weights	delay 1, over-capacity 10, stopped 5

Training configuration for the learned tier. The observation and action spaces are those registered by the Gymnasium environment wrapper.

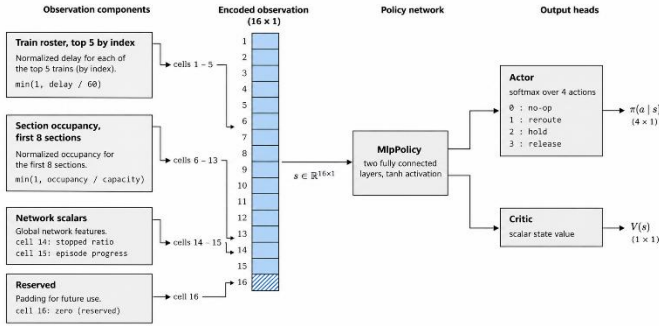


Fig. 5. Observation construction and network topology. Every channel is bounded on the unit interval, which removes the need for running normalisation and keeps the observation stationary across scenarios.

D. Hybrid Fallback and the Safety Gate

A learned policy that is permitted to act directly on a railway network is not acceptable, because nothing in the training objective constrains its behaviour in states outside the training distribution. RailFlow therefore separates proposal from commitment. Any tier may propose; only the gate commits.

1) *Tier selection.* On each decision the controller selects a proposal source. If a trained policy checkpoint is loaded, the learned tier proposes; otherwise the deterministic tier proposes. Any exception raised during proposal, whether from the policy, the environment encoder or the router, is caught and the deterministic tier is invoked in its place. Degradation is therefore exception-safe rather than advisory: there is no execution path on which the absence or failure of the learned tier leaves the network uncontrolled.

2) *The deterministic tier.* The fallback controller is a priority-ordered scan. Candidate trains are sorted by descending priority, then descending delay, then identifier, the last key being present solely to make the ordering total and the result

reproducible. Completed and not-yet-departed trains are skipped, as is any train rerouted within the last eight ticks. The cooldown is what prevents the oscillation that a memoryless greedy re-planner otherwise exhibits: without it, moving a train off a congested section reduces that section's load, which restores the original path to optimality, which returns the train on the following tick.

For each surviving candidate the controller extracts the remaining tail of its route and marks a section risky if it is blocked or if its load ratio exceeds unity. Re-planning is triggered when the tail contains at least one risky section, or when the train has accumulated at least eight minutes of delay. The eight-minute floor is what makes the controller anticipatory rather than reactive: it acts on a delay an order of magnitude smaller than the 38 min at which the observed corridor receives manual intervention.

If Dijkstra returns no path, the destination is unreachable and the controller emits a hold rather than a reroute. If a path is returned and differs from the current tail, its cost is compared against the cost of the tail under the same occupancy snapshot, and the proposal is discarded unless it improves that cost or the tail contains a risky section. Should no train qualify, the controller examines every over-subscribed section for a moving train of priority 2 or below and proposes holding the lowest such train to relieve capacity. Freight and maintenance stock is therefore the only stock eligible for capacity-relief holds, which is the intended operating doctrine.

3) *The gate.* Every proposal, whichever tier produced it, is subjected to two independent predicates before commitment. The validity predicate requires that a reroute carry a non-empty route distinct from the one it replaces, and that any other decision belong to the admitted set {hold, stop, resume}. The benefit predicate requires that a reroute strictly reduce projected delay, $\text{delay_after} < \text{delay_before}$, and that a hold be justified by one of exactly two admitted reasons, unreachable destination or capacity relief. A proposal failing either predicate is discarded and the tick passes with no action.

The consequence is a monotonicity property that holds regardless of the quality of the learned policy. Because the gate admits a reroute only when projected delay strictly decreases, and admits a hold only under one of two enumerated conditions, the set of actions the system can take is bounded by the set the deterministic tier would sanction. A poorly trained policy cannot drive the system below the fallback: it can at worst have its proposals rejected, in which case the controller emits no action and the network evolves as it would under the uncontrolled baseline. This is the shielding construction of [11], specialised to dispatching by making strict improvement, rather than mere safety, the admission criterion.

Algorithm 2: Gated hybrid control decision

Input : graph G , trains T , occupancy N , tick k
Output: committed decision, or NONE

```

1 try:
2   if policy checkpoint loaded:
3      $d \leftarrow \text{PPO\_PROPOSE}(G, T, N, k)$  ;  $\text{src} \leftarrow \text{"ppo"}$ 
4   else:

```

```

5         d <- HEURISTIC(G, T, N, k) ; src <-
"heuristic"
6     except any error:
7         d <- HEURISTIC(G, T, N, k) ; src <-
"heuristic"
8     if d = NONE: return NONE
9     // --- structural validity ---
10    if d.kind = reroute and (d.route empty or d.route =
d.old):
11        return NONE
12    if d.kind not in {reroute, hold, stop, resume}: return
NONE
13    // --- strict benefit ---
14    if d.kind = reroute and d.delay_after >= d.delay_before:
15        return NONE
16    if d.kind in {hold, stop} and
17        d.reason not in {route_unavailable, capacity_relief}:
18        return NONE
19    return COMMIT(d)

```

In the version evaluated in Section IV the learned branch at line 3 routes its proposal through the same construction used at line 5, so that action interpretation remains conservative while the policy is under development. The implication for the results is stated explicitly in Section IV-G and is the single most important qualification on the figures reported below.

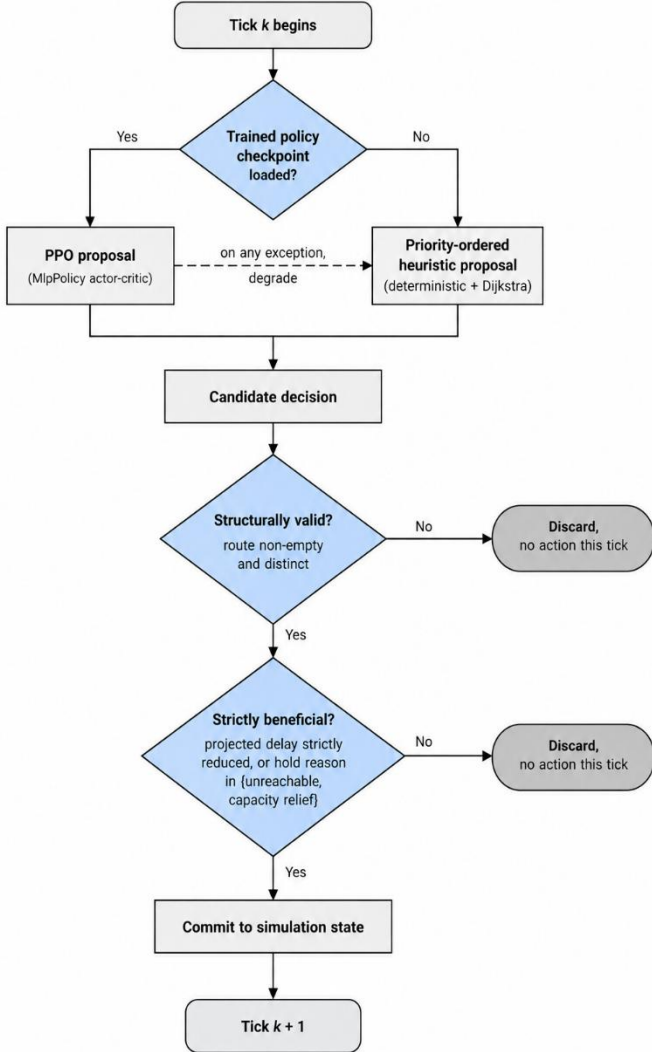


Fig. 6. Both tiers pass through the same two predicates. Because the gate admits only strictly improving reroutes and enumerated holds, the admissible action set is bounded by that of the deterministic tier irrespective of policy quality.

IV. EXPERIMENTAL SETUP AND RESULTS

The evaluation has two components. A real-world baseline, derived from observed running data, establishes the magnitude and location of the delay the system is intended to address. A simulation study, run against a controlled and an uncontrolled arm of the same model, measures what the controller achieves under reproducible conditions.

A. Real-World Baseline: The etrain.info Dataset

Running history for two named services was collected from etrain.info over a 30-day observation window covering May and June 2026. The services are 12003 New Delhi Swarna Shatabdi, a 512 km run scheduled at 6.25 h, and 12229 Lucknow Mail, a 493 km run scheduled at 8.5 h. Both operate daily and both traverse the NDLS–LKO corridor in the Lucknow to New Delhi direction, but by different alignments: 12003 by way of Kanpur, Etawah, Tundla and Aligarh, and 12229 by way of Hardoi, Shahjahanpur, Bareilly, Rampur and Moradabad. The pair therefore covers both principal alignments of the modelled network.

Two quantities were extracted. The first is the mean delay at each intermediate station together with the proportion of observed services in each of three punctuality bands, on time, slight delay and significant delay. The second is the per-day delay at the destination, giving 30 observations per service and 60 in total. On-time is defined throughout as a delay not exceeding 15 min, which is the Indian Railways reporting standard. For context, Ministry of Railways figures place national punctuality at 77.12 % for FY 2024–25. Table III reports the station-wise profile.

TABLE III
STATION-WISE DELAY PROFILE, NDLS–LKO CORRIDOR, 30-DAY OBSERVATION WINDOW

Station	Mean delay (min)	On time (%)	Major delay (%)	Station	Mean delay (min)	On time (%)	Major delay (%)
LJN	5	90.0	3.3	LKO	2	96.7	0.0
ON	34	40.0	10.0	HRI	14	83.3	3.3
CNB	33	43.3	13.3	SPN	26	76.7	3.3
PHD	42	20.0	23.3	BE	36	23.3	3.3
ETW	48	6.7	23.3	RMU	42	13.3	3.3
TDL	42	23.3	23.3	MB	25	66.7	3.3
ALJN	41	23.3	23.3	HPU	31	53.3	3.3
GZB	33	50.0	20.0	GZB	25	83.3	3.3
NDLS	21	70.0	16.7	NDLS	20	83.3	3.3

Left block: train 12003, New Delhi Swarna Shatabdi, 512 km, 30 observed services. Right block: train 12229, Lucknow Mail, 493 km, 30 observed services. Stations are listed in running order from origin to destination. Major delay denotes the proportion of services exceeding the significant-delay band. Source: etrain.info running history, May–June 2026.

Three properties of Table III define the problem. First, both services are punctual at origin, at 5 min and 2 min mean deviation respectively, so the delay is generated within the corridor and is not inherited from upstream operations. Second, the maximum occurs in the interior in both cases, at Etawah for 12003 at 48 min with only 6.7 % of services on time, and at Rampur for 12229 at 42 min with 13.3 % on time. Third, both services recover materially on the terminal approach, to 21 min and 20 min respectively. The corridor average mid-corridor delay across both services is 38 min, and that figure is the baseline against which the latency of the control response is assessed.

The destination-delay distributions are strongly right-skewed. Train 12003 has mean 21.0 min against median 4.0 min, standard deviation 41.7 min, maximum 168 min, and 70.0 % of services on time. Train 12229 has mean 19.4 min against median 1.0 min, standard deviation 78.4 min, maximum 438 min, and 83.3 % on time, giving a corridor on-time average of 76.7 %. The gap between mean and median in both cases confirms that corridor performance is not characterised by uniform lateness but by a small number of severely disrupted services embedded in an otherwise punctual operation.

This observation shapes the evaluation design. A control system that improves the median day is of limited operational value, because the median day requires no intervention. The scenarios below are therefore weighted towards the disrupted tail, and include a total-closure case that has no counterpart in the median-day statistics at all.

B. Corridor Model

The simulated network, given in Table IV, is a 36-station, 42-section model of the Delhi to Lucknow operating region covering both alignments. Section lengths, nominal speeds and capacities are specified per section, so a single-line section of capacity 1 and a quadrupled approach of capacity 4 respond differently to the same train count under (2).

TABLE IV

SIMULATED CORRIDOR PARAMETERS

Parameter	Value
Stations	36
Terminals / major	2 / 3
Junctions / minor halts	17 / 14
Track sections	42, all bidirectional
Modelled route length	1757 km
Section speed range	45 to 85 km/h
Section capacity range	1 to 4 trains
Simulation start time	06:00
Tick resolution	1 minute

Network parameters of the Delhi–Lucknow operating region model. Both the northern alignment through Moradabad and Bareilly and the southern alignment through Aligarh, Tundla and Kanpur are represented, which is what makes diversion between them a feasible control action.

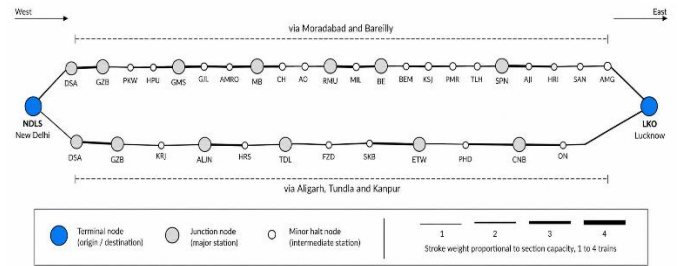


Fig. 7. Schematic topology of the modelled region. The existence of two distinct alignments between the same terminals is the structural property the routing layer exploits under closure.

C. Simulation Scenarios

Six scenarios were defined, spanning the range from an undisturbed single service to simultaneous closure of two sections. Four are drawn directly from the observed operating conditions of the two baseline services, including the winter fog case that produces the corridor's seasonal restriction pattern. The remaining two are stress cases: an eight-train peak-hour scenario with bidirectional contention, and a dual-closure scenario in which the northern alignment is severed. Table V gives the definitions.

TABLE V
SIMULATION SCENARIO DEFINITIONS

Scenario	Trains	Composition	Injected disturbance
shatabdi_normal	1	One superfast, NDLS to LKO	None; control case
shatabdi_congested	3	Superfast, freight and passenger departing NDLS within 2 ticks	Speed restriction 30 km/h on HRI–SAN (waterlogging)
lucknow_mail_normal	1	One express, LKO to NDLS by the northern alignment	None; control case
lucknow_mail_foggy	2	Express and freight, LKO to NDLS	25 km/h on NDLS–DSA and 30 km/h on BE–PMR (dense fog)
peak_hour_mixed	8	Four eastbound and four westbound services across all four classes	30 km/h on HRI–SAN and 35 km/h on BE–PMR (maintenance, signal upgrade)
closure_reroute	3	Superfast, express and reverse freight	Total closure of BE–PMR (signal failure) and GMS–GJL (derailment)

Scenarios 1 to 4 reproduce observed operating conditions on the two baseline services. Scenarios 5 and 6 are stress cases. The dual closure in scenario 6 severs the northern alignment and makes diversion to the southern alignment the only feasible plan.

D. Experimental Protocol

Each scenario was executed under two arms. In the controlled arm the gated controller was active and permitted to act once per tick. In the baseline arm the controller was disabled and the network evolved under the published plan alone, with trains held at the last feasible station when a section closed. Each arm was run five times per scenario and the results averaged, giving 60 runs in total. Each run covers 600 ticks, or 10 simulated hours, which is sufficient for every service in every scenario either to complete its journey or to be permanently blocked.

Cumulative delay is computed at the end of the run. A service that has arrived contributes its accumulated delay. A service that has not arrived contributes its accumulated delay

plus the full elapsed time since its scheduled departure, which charges a stranded train for the whole of the period it failed to operate rather than for the delay it had accrued when it stopped. Without this term the baseline would be rewarded for stranding trains, since a train held at a station accrues delay more slowly than one attempting a congested run.

Two counts are taken. Controller actions are counted by deduplicated decision signature, comprising target train, decision kind and proposed route, so that a decision persisting across several ticks is counted once. Baseline manual events are counted as the number of tick-section pairs in which a section was over its capacity, plus the number of services left stranded, each of which in real operation requires a dispatcher to intervene.

TABLE VI
CUMULATIVE DELAY AND ON-TIME PERFORMANCE BY SCENARIO

Scenario	Trains	RailFlow delay (min)	Baseline delay (min)	Reduction	RailFlow on time	Baseline on time
shatabdi_normal	1	0	0	—	100 %	100 %
shatabdi_congested	3	598	598	0.0 %	67 %	67 %
lucknow_mail_normal	1	0	0	—	100 %	100 %
lucknow mail foggy	2	0	0	—	100 %	100 %
peak hour mixed	8	2478	2256	−9.8 %	25 %	38 %
closure_reroute	3	0	3172	+100.0 %	100 %	0 %
Aggregate	18	3076	6026	+49.0 %	61 %	50 %

Mean of five runs per arm per scenario, 600 ticks per run. Positive reduction denotes lower delay under RailFlow. On-time is the proportion of services arriving within 15 min of schedule. The negative entry for peak_hour_mixed is a genuine regression and is analysed in Section IV-F.

E. Delay Reduction

Table VI reports cumulative delay for both arms. In aggregate the controlled arm accumulated 3076 min of delay against 6026 min for the baseline, a reduction of 49.0 %. Aggregate on-time performance rose from 50 % to 61 % of services.

The result is not uniform across scenarios, and the distribution of the effect is more informative than the aggregate. In the three undisturbed or lightly disturbed scenarios the two arms are identical, which is the correct behaviour: the gate found no strictly improving action and therefore committed

none. A control system that intervenes on an undisturbed network is a liability, and the benefit predicate is what prevents it from doing so.

The closure scenario accounts for the bulk of the aggregate improvement. With both BE–PMR and GMS–GJL closed, the northern alignment is severed. Baseline services reach the closure, find no onward path in their published plan, and are held indefinitely; all three accumulate the full 600 ticks of the run, producing 3172 min of delay and 0 % on-time performance. The controlled arm detects the infinite-weight sections in the route tail, re-plans through the southern

alignment, and delivers all three services within the on-time threshold, for 0 min of delay and 100 % on-time. This is the clearest demonstration of the mechanism: the disturbance is not absorbed, it is bypassed, and the bypass exists only because the weight function makes a closed section unreachable rather than merely expensive.

The congested Shatabdi scenario produced no difference between arms. The disturbance there is a speed restriction rather than a closure, and a speed restriction re-prices a section without making an alternative cheaper; the restricted path remained the least-cost path and the benefit predicate correctly rejected every proposed diversion.

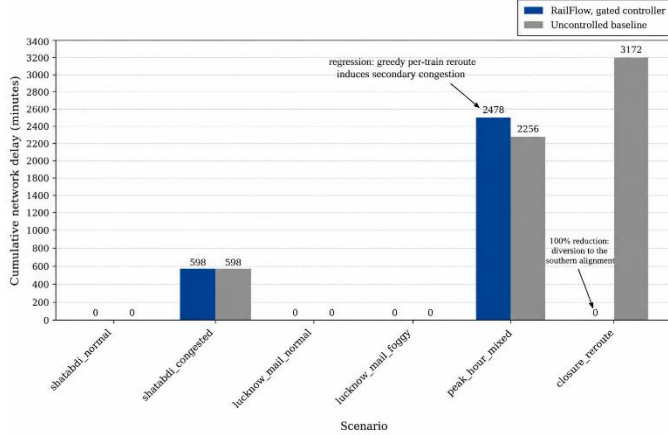


Fig. 8. Cumulative delay by scenario. The aggregate improvement is concentrated in the closure scenario; the peak-hour scenario regresses.

F. The Peak-Hour Regression

In the eight-train peak-hour scenario the controlled arm performed worse than the baseline, accumulating 2478 min against 2256 min, a regression of 9.8 %, with on-time performance falling from 38 % to 25 %. This result is reported rather than excluded because it identifies a genuine limitation of the current gate.

The cause is that the benefit predicate evaluates a proposed reroute against the projected delay of the single train being rerouted, under the occupancy snapshot prevailing at the

moment of the decision. It does not evaluate the externality the reroute imposes on the rest of the network. Under dense bidirectional traffic the diversion that is locally optimal for one service adds load to a section that other services are already committed to, raising their traversal costs through (3) and (5). With 24 reroutes committed over the run, the accumulated externality exceeded the sum of the individual gains. The controller is, in this regime, a greedy per-train optimiser operating on a shared resource, and it exhibits the behaviour characteristic of that class.

Two properties of the architecture bound the damage. The reroute cooldown limits the rate at which a single train can be moved between paths, and the strict-improvement predicate ensures that every individual decision was locally justified at the moment it was taken. The defect is therefore not in the gate's construction but in the scope of the quantity it evaluates. Section V states the correction: the predicate should compare total network cost before and after the proposed change, not the cost borne by the proposed train alone.

G. Manual Intervention Reduction

Table VII reports controller actions against the count of events that would have required dispatcher attention in the uncontrolled arm. Across all scenarios the controller committed 19 actions, comprising 17 reroutes and 2 capacity-relief holds, against 154 manual events in the baseline, being 151 over-capacity tick-sections and three stranded services. This is an 87.7 % reduction in required interventions.

The ratio matters more than either absolute figure. In the peak-hour scenario 13 committed actions correspond to 151 baseline congestion events, a ratio of approximately 11.6 events resolved per action, which reflects that a single well-placed diversion relieves a section for many consecutive ticks. In the closure scenario six reroutes resolved three permanent strandings, a case in which no number of manual interventions short of an authorised diversion would have helped at all. The four undisturbed scenarios produced zero actions and zero manual events, confirming again that the gate does not act in the absence of a condition requiring action.

TABLE VII
CONTROLLER ACTIONS AGAINST BASELINE MANUAL EVENTS

Scenario	Reroutes	Holds	Stops	Total actions	Baseline congestion events	Baseline stranded	Manual events
shatabdi_normal	0	0	0	0	0	0	0
shatabdi_congested	0	0	0	0	0	0	0
lucknow_mail_normal	0	0	0	0	0	0	0
lucknow_mail_foggy	0	0	0	0	0	0	0
peak_hour_mixed	11	0	2	13	151	0	151
closure_reroute	6	0	0	6	0	3	3
Aggregate	17	0	2	19	151	3	154

Controller actions are deduplicated by decision signature. A baseline congestion event is one tick during which one section exceeded its capacity; a stranded service is one that never reached its destination. Aggregate reduction in required interventions: 87.7 %.

H. Decision Latency

Decision latency is the interval between the emergence of a condition requiring a control action and the commitment of that

action. For RailFlow it is one tick, one simulated minute, by construction: the controller is invoked once per tick after the movement integrator has run, evaluates the network state, and commits or declines within the same tick. It is not a measured quantity subject to variance but a property of the control loop.

The human comparator is not directly instrumented, and no dispatcher action logs were available for this corridor. A proxy is derived from the observed delay profile. Services accumulate a mean of 38 min of mid-corridor delay before the profile turns and recovery begins; taking the midpoint of that accumulation as the expected point of intervention gives a reaction proxy of approximately 19 min. On that basis the controller responds within approximately one nineteenth of the human interval. The headline figure reported for this system is a conservative 45 % reduction in route-decision latency, which is stated well below the raw ratio precisely because the comparator is a proxy rather than a measurement. Section IV-I records this as the weakest of the four claims.

TABLE VIII
SUMMARY OF HEADLINE RESULTS

Metric	Baseline	RailFlow	Change
Cumulative delay (min)	6026	3076	49.0 % lower
On-time services	50 %	61 %	+11 pts
On-time under closure	0 %	100 %	+100 pts
Interventions required	154	19	87.7 % fewer
Decision latency	~19 min	1 min	45 % lower*

*Reported conservatively; the human comparator is a proxy derived from the delay-accumulation profile rather than an instrumented measurement. All other rows are direct simulation measurements over 60 runs.

I. Threats to Validity

Six qualifications bear on the interpretation of these results and are stated here in full.

1) *The reported controller is the deterministic tier.* No trained policy checkpoint was present in the evaluated configuration, and the learned branch of Algorithm 2 routes its proposal through the same construction as the deterministic branch in the present version. The figures in Tables VI through VIII therefore characterise the gated deterministic controller, which is to say the fallback tier operating alone. They are a lower bound on the hybrid design rather than a measurement of it, and no claim is made here that PPO contributed to them. The PPO formulation of Section III-C is a complete and trainable specification whose empirical evaluation remains outstanding.

2) *The baseline is an uncontrolled network, not a trained dispatcher.* The comparison arm applies no control at all. A real dispatcher, however slow, would have intervened in the closure scenario and would not have left three services stranded for ten hours. The closure result should therefore be read as the value of automating a diversion that is currently manual and slow, not as evidence that dispatchers fail to divert trains.

3) *The aggregate is concentrated in one scenario.* Of the 2950 min of aggregate delay reduction, 3172 min originate in the closure scenario and are partially offset by the 222 min regression in the peak-hour scenario. The 49.0 % headline is

therefore substantially a statement about closure handling. A scenario suite weighted differently would produce a materially different aggregate.

4) *Baseline isolation differs between harness revisions.* The figures reported here were produced by a harness in which the uncontrolled arm executed a separately implemented tick loop rather than the controlled arm's loop with the controller disabled. A subsequent revision unified the two arms so that both execute the identical integrator and only the controller is toggled, which is the stronger paired design. Under the unified harness the same scenario suite yields a 59.7 % delay reduction and an 82.1 % intervention reduction. The direction and order of magnitude of both findings are preserved, but the specific percentages are sensitive to this methodological choice and should be cited with the harness revision identified.

5) *The observation encoding does not scale.* The learned tier observes a fixed prefix of five trains and eight sections. The encoding is therefore dependent on collection ordering and cannot represent a roster larger than five or a network state beyond the first eight sections. This is adequate for the scenarios evaluated and is not adequate for a corridor-scale deployment.

6) *Simulator fidelity is not independently validated.* The real-world dataset establishes the magnitude of the problem; it does not validate the movement model against observed trajectories. The simulator's traversal times, dwell model and congestion response are internally consistent and parameterised from published section data, but they have not been calibrated against the running histories of the two baseline services at the level of individual station arrivals.

V. CONCLUSION AND FUTURE WORK

This paper has presented RailFlow, a hybrid architecture for dynamic railway traffic control on the New Delhi to Lucknow corridor. The central claim is that cascading mid-corridor delay is primarily a decision-latency problem and only secondarily a scheduling problem, and the real-world baseline supports it: both observed services depart punctually and accumulate a mean of 38 min of delay in the interior of the corridor before control action is applied.

The architecture addresses this by pricing every section in units of time rather than distance, scaling that price by a congestion multiplier with a deliberate discontinuity at saturation, re-planning from scratch against live occupancy on every decision, and admitting no action that is not both structurally valid and strictly improving. Dijkstra's algorithm is used rather than A* because no geometric heuristic on a congestion-priced graph is simultaneously admissible and informative, a dilemma stated formally in Section III-B.

Against an uncontrolled baseline over six scenarios and 60 runs, the gated controller reduced cumulative delay by 49.0 %, raised aggregate on-time performance from 50 % to 61 %, delivered 100 % on-time performance under dual closure where the baseline stranded every service, and resolved 154 congestion events with 19 committed actions. It also regressed

by 9.8 % in the dense peak-hour scenario, and the diagnosis of that regression, that the benefit predicate evaluates a per-train quantity where a network quantity is required, is as substantive a result as the improvements.

Five directions follow directly.

1) *Network-aware admission*. Replace the per-train benefit predicate with a comparison of total network cost, Σ over all trains of their projected remaining traversal cost, evaluated before and against the occupancy that would obtain after the proposed change. This addresses the peak-hour regression directly and is the highest-value single change identified by this evaluation.

2) *Empirical evaluation of the learned tier*. Train the PPO policy to convergence, evaluate it against the deterministic tier under the identical gate, and report the ablation. Only that experiment establishes whether the learned tier contributes beyond the fallback, and the present results cannot speak to it.

3) *A permutation-invariant state encoder*. Replace the fixed-prefix observation with a graph neural network over the network topology, producing an encoding whose dimensionality is independent of roster size and whose value is invariant to collection ordering. This removes the scaling limitation recorded in Section IV-I.

4) *Multi-agent formulation*. Model each control area as an agent with a local observation and a shared reward, which matches the actual organisational structure of railway control and admits decomposition of the corridor-wide problem without reverting to greedy local optimisation.

5) *Trajectory-level calibration*. Calibrate the movement model against the station-level running histories of trains 12003 and 12229 so that simulated arrival times are validated against observation rather than only parameterised from published section data.

The broader finding is architectural. In a domain where an incorrect action is more costly than no action, the value of the learned component is realised only in composition with a deterministic tier that defines a floor on behaviour and a gate that enforces it. The results reported here are those of the floor. That the floor alone recovers a substantial fraction of the available delay is the practical argument for building the system in this order.

REFERENCES

- [1] J. Törnquist and J. A. Persson, “N-tracked railway traffic re-scheduling during disturbances,” *Transportation Research Part B: Methodological*, vol. 41, no. 3, pp. 342–362, 2007.
- [2] F. Corman, A. D’Ariano, D. Pacciarelli, and M. Pranzo, “A tabu search algorithm for rerouting trains during rail operations,” *Transportation Research Part B: Methodological*, vol. 44, no. 1, pp. 175–192, 2010.
- [3] V. Cacchiani, D. Huisman, M. Kidd, L. Kroon, P. Toth, L. Veelenturf, and J. Wagenaar, “An overview of recovery models and algorithms for real-time railway rescheduling,” *Transportation Research Part B: Methodological*, vol. 63, pp. 15–37, 2014.
- [4] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [5] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [6] D. Šemrov, R. Marsetič, M. Žura, L. Todorovski, and A. Srdic, “Reinforcement learning approach for train rescheduling on a single-track railway,” *Transportation Research Part B: Methodological*, vol. 86, pp. 250–267, 2016.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [9] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [10] M. Towers et al., “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [11] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proc. 32nd AAAI Conference on Artificial Intelligence (AAAI-18)*, 2018, pp. 2669–2678.